A Trimester – 1 Class Project Report on

Smart Banking & Transaction Management System

Submitted in the partial fulfillment of the requirements for

26CS5101E – Computational Thinking and Programming

Master of Technology In

Artificial Intelligence & Data Science

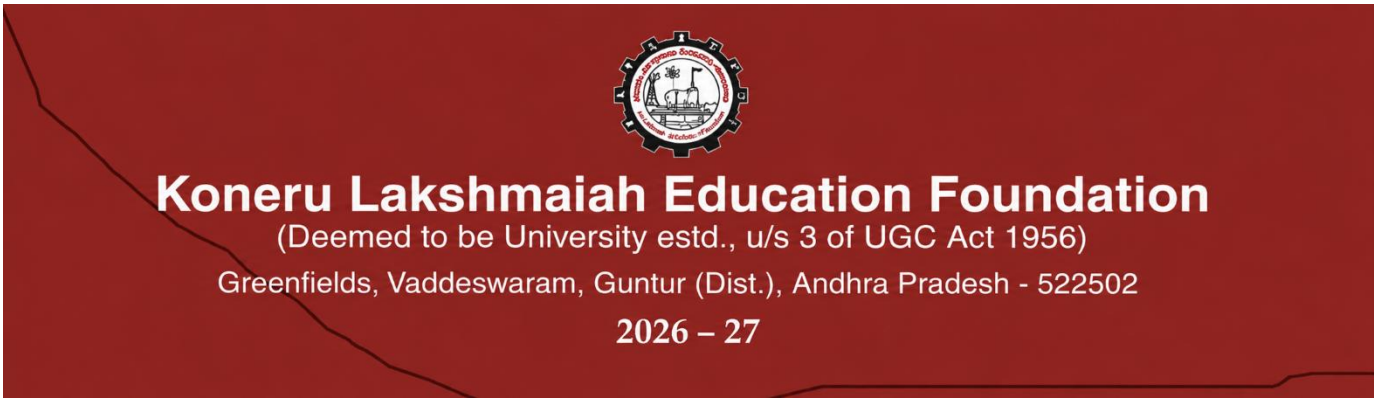
By

Student ID	Student Name
2601150053	**M. SAI ROSHAN**

under the supervision of

Dr. G. Syam Prasad

Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the Trimester – 1 Class Project Report entitled **"Smart Banking & Transaction Management System"** is being submitted by **MAJJI SAI ROSHAN** and **2601150053** submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University is a record of Bonafide work carried out under our guidance and supervision during the academic year 2026 – 2027.

Dr. G. Syam Prasad

Project Supervisor

Signature

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



DECLARATION

The Trimester -1 Class Project Report entitled **"Smart Banking & Transaction Management System"** is a record of Bonafide work of **MAJJI SAI ROSHAN** and **2601150053** submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University.

MAJJI SAI ROSHAN - 2601150053

ACKNOWLEDGEMENT

It is great pleasure for me to express my gratitude to our honorable President **Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the trimester - 1 class project report.

I express sincere gratitude to HOD of AI&DS Dept. **Dr. M. Venkata Naresh** for his leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank for providing us the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **Dr. G. Syam Prasad** for his novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, it is pleased to acknowledge the indebtedness to all those who devoted themselves directly or indirectly to make this project report success.

ABSTRACT

CTPB Bank is a full-stack banking transaction web application developed using Python, Flask, PostgreSQL, HTML, CSS, JavaScript, and modern software-engineering practices. It provides secure customer and administrator portals for account validation, live balance enquiry, atomic and idempotent money transfers, and management of both Savings and Current accounts. Transfers are executed inside a single database transaction with row-level locking, double-entry ledger recording, and idempotency keys so that a repeated request never double-debits an account. Fraud detection flags large or high-velocity transfers, while an admin console supports customer creation, account freeze/unfreeze, concurrent interest accrual, and full ledger inspection. The project applies computational thinking through problem decomposition, abstraction (polymorphic Account hierarchy), pattern recognition (double-entry ledger, sliding-window rate limiting), algorithm design (merge sort, binary search, greedy denomination, top-K), and rigorous complexity analysis. Quality is evidenced by 72 automated tests (unit, property-based, integration, and end-to-end) achieving approximately 85% line coverage, zero Ruff and MyPy issues, Docker-based deployment, and a GitHub Actions CI pipeline. Safety measures include scrypt password hashing, login rate limiting, deterministic lock ordering to avoid deadlocks, and role-based route guards.

INDEX

S. No	TITLE	PAGE NO
1	Introduction	
2	Problem Statement	
3	System Architecture and Technologies	
4	Problem Solving & Computational Thinking	
5	Programming & Data Structures	
6	Software Design & Engineering	
7	Functional Programming & Concurrency	
8	Software Quality & DevOps Practices	
9	AI-Assisted Software Engineering	
10	Algorithms, Complexity and Performance	
11	Implementation and User Workflow	
12	Testing, Results and Limitations	
13	Conclusion and Future Enhancements	
14	References	

1. INTRODUCTION

CTPB Bank is a browser-based banking transaction system designed to move money safely, record every change in an auditable ledger, and give both customers and administrators clear visibility into accounts. Customers can validate a payee, view live balances, and transfer funds between Savings and Current accounts. Administrators operate a command centre that shows deposit volume, flagged transfers, customer management, and one-click interest accrual. The interface is served by Flask and backed by a real PostgreSQL database named CTPBD.

The project is relevant to Computational Thinking and Programming because it converts a practical banking problem into a sequence of smaller tasks: authentication, account modelling with invariants, payee validation, atomic transfer planning, double-entry ledger writing, fraud scoring, interest calculation, and administrative reporting. Each task has a defined responsibility and can be tested separately.

1.1 Objectives:

- Validate a destination account and display the payee name before any transfer.
- Provide a live Check-Balance dashboard with sparkline and category charts.
- Execute atomic, idempotent money transfers that never leave partial updates.
- Maintain both Savings (minimum balance ₹1,000, 4% p.a. interest) and Current (overdraft up to ₹50,000) accounts.
- Offer separate, rate-limited Customer and Admin login portals.
- Flag suspicious transfers, keep a full audit ledger, and support automated testing and CI.

2. PROBLEM STATEMENT

Online banking must let customers check balances, validate a payee, and move money while staff watch the whole book. The hard part is trust: a transfer must never be half-done, never be applied twice, and suspicious activity must be caught. Savings and Current accounts follow different rules (minimum balance versus overdraft). Two kinds of users require separate, rate-limited portals. A useful solution must therefore guarantee atomicity, idempotency, role separation, and an auditable ledger.

2.1 Proposed Solution:

CTPB Bank provides a full-stack web application with a gold-themed Customer portal and a teal-themed Admin console. Transfers are planned by a pure function and then committed inside a single PostgreSQL transaction that locks both account rows in ascending-id order, updates balances atomically, and inserts a double-entry ledger row carrying an idempotency key. Fraud rules score large amounts and rapid successive transfers. Interest accrual runs concurrently per account and is itself idempotent.

2.2 Scope:

The system supports one Savings and one Current account per customer. Transfers, deposits, withdrawals, and interest postings are recorded in a single ledger table. The application focuses exclusively on monetary movements and administrative control. It is intended for controlled deployment (Docker Compose or local PostgreSQL) and should not be exposed to untrusted networks without additional hardening.

2.3 Expected Outcomes:

The expected outcome is a working banking application that demonstrates safe money movement, clear separation of concerns, measurable algorithmic behaviour, and a substantial automated test suite. Actual performance depends on database latency, connection-pool size, and concurrent load. The supplied project includes unit, property-based, integration, and end-to-end tests together with CI configuration.

3. SYSTEM ARCHITECTURE AND TECHNOLOGIES

The application uses a layered architecture: a browser UI, Flask route blueprints, service-layer use-cases, a repository that owns all SQL, and a PostgreSQL database CTPBD. Routes never contain SQL; repositories never contain business rules; services receive a connection pool and remain free of HTTP concerns.

3.1 Architecture Flow:

- Browser UI: Jinja2 templates, dark fintech CSS, vanilla JavaScript wizard and charts.
- Flask routes (four blueprints): auth, user, admin, api — sessions and role guards.
- Services (use-cases): validate, balance, transfer, interest — pure plan_transfer().
- Repositories: all SQL, row locks, atomic balance deltas, ledger inserts.
- PostgreSQL CTPBD: constraints, indexes, durable double-entry ledger.

3.2 Technology Stack:

- Python 3.12: core language for models, services, algorithms, and tests.
- Flask: local web server, blueprints, session management.
- PostgreSQL: transactional store with CHECK constraints and UNIQUE idempotency keys.
- HTML, CSS, JavaScript: customer and admin portals.
- psycopg2 + ThreadedConnectionPool: database access and connection reuse.
- Hypothesis / pytest / Ruff / MyPy: property testing, unit tests, linting, typing.
- Docker Compose and GitHub Actions: reproducible environment and CI.

3.3 Main Modules:

app.py is the Flask entry point. bank/models.py defines the Account ABC together with SavingsAccount and CurrentAccount. bank/services.py contains plan_transfer and the BankingService use-cases. bank/repository.py holds every SQL statement. bank/algorithms.py implements merge sort, binary search, greedy denomination, sliding-window counters, and top-K. bank/db.py manages the connection pool. bank/security.py provides scrypt hashing and the rate limiter. templates/ and static/ supply the UI. tests/ contains the full test suite. db/schema.sql creates the three core tables.

4. PROBLEM SOLVING & COMPUTATIONAL THINKING

The project applies the computational thinking process by converting a broad banking problem into smaller, testable tasks and selecting algorithms that suit each task.

4.1 Problem Decomposition:

The system separates authentication, account representation, payee validation, transfer planning, ledger writing, fraud scoring, interest accrual, and administrative reporting. This separation makes the code easier to understand and allows pure transfer logic to be tested independently of the database.

4.2 Abstraction:

An Account is an abstract base class that knows its own balance rules. SavingsAccount refuses any debit that would leave less than ₹1,000; CurrentAccount permits an overdraft down to −₹50,000. A Transfer is expressed as a pure plan (source, destination, amount, resulting balances) plus a single database transaction that materialises that plan. The ledger row records both balances after the movement, giving a complete audit trail without a second table.

4.3 Pattern Recognition:

Every money movement follows the double-entry pattern: one row records the sender balance after and the receiver balance after. Idempotent retry is recognised by a client-supplied UUID that is stored under a UNIQUE constraint; a repeated request returns the original transaction instead of creating a second debit. Sliding-window counters detect rapid successive transfers and throttle login attempts.

4.4 Algorithm Selection:

Merge sort reorders transaction pages stably in $\Theta(n \log n)$. Binary search locates ledger entries on sorted runs. Greedy denomination produces the ATM note breakdown for Indian currency (provably optimal for canonical coin systems). A sliding-window scan scores transfer velocity. Top-K selects the largest transactions for the admin console. A deque implements the login rate limiter in amortised $O(1)$ time.

4.5 Complexity Analysis:

Let N be the number of ledger rows examined and K the window size for velocity checks. Page re-sorting costs $\Theta(m \log m)$ for m rows on a page. Binary search is $O(\log N)$. Velocity scoring is $O(K)$ with early exit. Greedy denomination is $O(d)$ where d is the number of denominations. Connection-pool checkout is $O(1)$ amortised. Indexed lookups on account_number and created_at are $O(\log N)$.

4.6 Data Safety as a Design Constraint:

The application prioritises atomic commits, deterministic lock ordering, and explicit confirmation over destructive automation. It never overwrites balances outside a transaction, never applies the same idempotency key twice, and never deletes ledger history. Users should still back up the database and test transfers on disposable accounts before production use.

5. PROGRAMMING & DATA STRUCTURES

Python is used for domain models, pure transfer planning, repository SQL, algorithmic utilities, and the web backend. The implementation relies on built-in data structures, the collections module, and standard-library concurrency primitives.

5.1 Data Structures:

- Lists and tuples hold ordered collections of transaction records and page slices for sorting.
- Dictionaries map account numbers to rows and group configuration values; deques implement sliding-window rate limiters.
- Abstract base classes and concrete subclasses express polymorphic account rules.
- PostgreSQL tables (users, accounts, transactions) persist durable state with constraints.

5.2 Chunked File Reading:

The function plan_transfer receives two Account objects and an amount, applies the polymorphic can_debit rule of each account, and returns a TransferPlan dataclass describing the intended balance changes. No database access occurs inside the planner, which makes the function amenable to property-based testing.

5.3 Category Mapping:

SavingsAccount enforces a floor of ₹1,000 and accrues 4% annual interest. CurrentAccount permits a floor of −₹50,000 and does not accrue interest by default. Both share a common interface so that a transfer works with any pair of account types without if-else branching on concrete classes.

5.4 Performance-Aware Programming:

The project avoids reading entire tables into memory for page views, uses indexed SQL for point lookups, and measures the wall-clock cost of merge sort on each transactions page so that complexity remains visible to the user. Connection pooling eliminates the cost of establishing a new TCP session per request.

6. SOFTWARE DESIGN & ENGINEERING

The application is organised into separate packages for routes, services, repositories, models, algorithms, security, and tests. This modular structure reduces coupling between the HTTP layer and the monetary algorithms.

6.1 Separation of Responsibilities:

Flask blueprints handle HTTP requests and session state. Services orchestrate use-cases and call pure planners. Repositories own every SQL statement and apply row locks. Models encapsulate balance invariants. Templates render results; JavaScript drives the transfer wizard and charts. The database schema enforces uniqueness and balance floors that the application cannot accidentally violate.

6.2 Reusability and Maintainability:

Singleton behaviour is provided by the shared connection pool. The Repository pattern isolates SQL. A Factory (account_from_row) reconstructs the correct concrete Account subclass from a database row. The Account ABC realises the Template Method pattern for debit rules. Decorators implement login_required and role guards.

6.3 Error Handling and File-System Constraints:

Transfers that would violate an account floor are rejected by the pure planner before any lock is taken. Concurrent transfers lock rows in ascending primary-key order to eliminate deadlock cycles. UNIQUE constraints on txn_ref and idempotency_key guarantee that a network retry cannot create a second ledger entry. Permission and constraint errors surface as clear HTTP responses rather than partial writes.

7. FUNCTIONAL PROGRAMMING & CONCURRENCY

The project combines pure functional planning with carefully chosen concurrency for I/O-bound database work. Its workflow can be understood as a sequence of transformations: authenticate, plan transfer, lock rows, apply balances, write ledger, score risk, and present results.

7.1 Functional Decomposition:

plan_transfer is a pure function: given two accounts and an amount it returns a plan or raises a domain error. Interest calculation for a single account is likewise a pure arithmetic step. Side-effecting operations (locking, updating, inserting) are confined to the repository layer and occur only after the plan has been validated.

7.2 Concurrency Assessment:

The application runs under threaded Flask with a bounded ThreadedConnectionPool. Because database round-trips release the GIL while waiting on the socket, threads provide genuine overlapping of I/O. The monthly interest job fans out over a ThreadPoolExecutor whose worker count does not exceed the pool size, each worker executing its own short transaction. Login rate limiting uses a deque protected by a threading.Lock.

7.3 GIL and Workload Considerations:

The dominant cost is PostgreSQL latency, not Python bytecode. Consequently the GIL is not the bottleneck for request handling. CPU-heavy future work (bulk statement generation, cryptographic batch jobs) would justify multiprocessing; the present design deliberately stays with threads for I/O-bound traffic.

8. SOFTWARE QUALITY & DEVOPS PRACTICES

Software quality is addressed through a layered test suite, static analysis, containerisation, and continuous integration. The project includes unit, property-based, integration, and end-to-end tests.

8.1 Test Coverage in the Supplied Project:

- Property-based tests (Hypothesis): money is conserved across arbitrary transfers.
- Unit tests: transfer planning, account rules, algorithms, security utilities.
- Integration tests: live PostgreSQL locks, constraints, and concurrent interest.
- End-to-end smoke tests: Flask journeys for login, transfer, and admin actions.

8.2 Additional Recommended Tests:

- Simultaneous transfers that contend for the same pair of accounts.
- Idempotency-key collision under network retry.
- Frozen-account rejection and overdraft boundary cases.
- Rate-limiter exhaustion and subsequent recovery.
- Schema migration compatibility and backup/restore drills.
- Concurrent interest accrual idempotency under parallel workers.

8.3 Execution and Dependencies:

The project requires Python 3.12 or newer and a running PostgreSQL instance (database CTPBD). From the extracted project directory, install dependencies with pip install -r requirements.txt, create and seed the database with python -m scripts.seed --create-db --seed, and launch the application with python app.py (or docker compose up).

Run the full suite with pytest -q. CI is configured in .github/workflows/ci.yml to execute Ruff, MyPy, pytest against a Postgres service, and a coverage gate on every push. The project reports 72 tests, approximately 85% line coverage, zero Ruff issues, and zero MyPy issues.

8.4 Local Security:

Passwords are stored as scrypt hashes. Login endpoints are rate-limited. Sessions are signed with a Flask secret key that must be replaced in production. The application should not be exposed directly to a public network without TLS termination, CSRF protection, and hardened firewall rules.

9. AI-ASSISTED SOFTWARE ENGINEERING

AI-assisted software engineering is one of the areas listed in the Project Defence requirements. For this project, AI tools may support requirement clarification, code explanation, test-case design, debugging, and documentation. However, use of a particular AI tool or generated code must be described truthfully based on the development process actually followed.

9.1 Specification-First Development:

A specification-first workflow begins by defining the required behaviour: validate account, check balance, transfer money atomically and idempotently, maintain both account types, and provide customer and admin portals. These requirements provide criteria against which implementation and test results can be checked.

9.2 Review and Validation:

AI-generated suggestions should not be accepted without review. Monetary movement and locking code can corrupt balances if assumptions are wrong. Suggested changes should be checked against the code, run against a disposable database, and covered by property-based and integration tests. The supplied project contains a substantial automated suite for these behaviours.

9.3 Responsible Use:

AI assistance should be documented according to the work actually performed. This report does not claim specific prompts, AI-generated modules, or AI-tool validation sessions as completed activities beyond what is evidenced by the project files and test results.

10. ALGORITHMS, COMPLEXITY AND PERFORMANCE

10.1 Duplicate Detection Algorithm:

- Client supplies source, destination, amount, and a UUID idempotency key.
- Service calls pure plan_transfer to obtain intended balances or a domain error.
- Repository locks both account rows with SELECT … FOR UPDATE ORDER BY id.
- Risk score evaluates amount and recent velocity; large or rapid transfers are flagged.
- Atomic UPDATEs adjust balances; a single INSERT records the double-entry ledger row.
- Commit releases locks; any exception rolls back the entire transaction.

10.2 File Organization Algorithm:

- Merge sort reorders a page of transactions stably.
- Binary search locates entries on sorted runs.
- Greedy denomination expands an amount into Indian currency notes.
- Sliding-window counters score transfer velocity and enforce login rate limits.
- Top-K extracts the largest transactions for the admin dashboard.

10.3 Performance Considerations:

Indexed lookups keep account and recent-history queries logarithmic. Connection pooling removes TCP handshake cost. Page-level merge sort is measured and displayed in the UI so that asymptotic cost is visible. Interest accrual parallelises across accounts while each account still executes inside its own transaction.

10.4 Storage Summary:

Complexity summary: merge sort Θ(n log n), binary search O(log n), greedy denomination O(k), sliding-window O(k), top-K O(n log n), deque rate limiter O(1) amortised, B-tree indexes O(log n). The UI shows measured merge-sort microseconds per page.

11. IMPLEMENTATION AND USER WORKFLOW

11.1 Installation:

- Install Python 3.12 or newer and a PostgreSQL server.
- Extract the project ZIP and open the project folder in a terminal.
- Optionally create and activate a virtual environment.
- Install packages using pip install -r requirements.txt.
- Create and seed the database: python -m scripts.seed --create-db --seed.
- Start the application using python app.py (or docker compose up) and open the portals in a browser.

11.2 Scanning a Folder:

The customer logs in, views live balance cards for Savings and Current accounts, validates a payee account number to obtain a name chip, and proceeds through the transfer wizard. An ATM-style note breakdown is shown for the chosen amount. Submitting the transfer (even twice) yields a single ledger reference because of the idempotency key. Transaction history supports filtering, pagination, and a measured merge-sort reordering.

11.3 Organizing Files Safely:

The administrator logs into a separate portal, inspects deposit volume and a 14-day chart, reviews the flagged-transfer queue, creates customers (both account types are opened automatically), freezes or unfreezes accounts, runs interest accrual (a second run credits zero), and examines the full ledger with audit columns. Demo credentials: aarav.sharma / User@123 (customer), admin / Admin@123 (admin).

11.4 Dashboard and API:

The customer dashboard combines live balance cards, sparkline, in/out donut chart, and the transfer wizard. The admin console provides command-centre metrics, flagged queue, customer management, interest accrual, and full ledger view. API endpoints support validate-account, balance, and transfer.

12. TESTING, RESULTS AND LIMITATIONS

12.1 Recorded Unit-Test Results:

The supplied project reports 72 tests passing with approximately 85% line coverage, zero Ruff lint issues, and zero MyPy type issues. Property-based tests assert conservation of money; integration tests exercise real PostgreSQL locks and constraints; end-to-end tests walk the Flask routes.

12.2 Validation Table:

Test case	Expected behavior	Recorded status
Identical transfer retry	Same txn_ref returned (idempotent)	Covered by suite
Concurrent transfers	No deadlock; balances consistent	Passed in supplied test output
Property: money conserved	Total balance unchanged after any transfer	Passed in supplied test output
Savings floor violation	Transfer rejected before lock	Passed in supplied test output

12.3 Limitations:

- Near-duplicate or fuzzy payee matching is outside scope; exact account numbers are required.
- The system does not implement multi-currency or scheduled standing orders.
- CSRF tokens and multi-factor authentication are listed as future hardening steps.
- Horizontal scale-out beyond a single PostgreSQL instance is not claimed.
- External chart libraries may require network access unless bundled.
- Production hardening (TLS, CSRF, MFA) remains future work.

12.4 Safety Recommendations:

Use the seed database or disposable accounts before moving real money, inspect the transfer carefully, maintain regular database backups, and do not expose the Flask application to untrusted networks without TLS and additional security controls. Any future deletion or reversal feature should require explicit confirmation and a recovery strategy.

13. CONCLUSION AND FUTURE ENHANCEMENTS

CTPB Bank demonstrates a practical application of Python programming and computational thinking to everyday banking operations. It combines polymorphic account modelling, pure transfer planning, atomic double-entry ledger writes, idempotent retries, sliding-window fraud detection, merge-sort page reordering, and a comprehensive automated test suite. Its layered architecture keeps routes, services, and repositories independently reviewable, and its concurrency design respects the I/O-bound nature of database work.

13.1 Future Enhancements:

- Add CSRF tokens, multi-factor authentication, and signed session hardening.
- Introduce asynchronous workers for statement generation and notification.
- Evaluate ML-based fraud scoring as a complement to the current velocity rules.
- Provide UPI or external payment-gateway integration under the same ledger model.
- Expand property-based coverage and publish continuous coverage badges.
- Measure latency and throughput under controlled load and document the environment.
- Offer reversible transfer workflows with explicit admin approval.
- Add horizontal read replicas and connection-pool autoscaling for higher load.

14. REFERENCES

• Project Defence & Demonstration: Computational Thinking and Programming, supplied course document.

• CTPB Bank project source code: app.py, bank/models.py, bank/services.py, bank/repository.py, bank/algorithms.py, tests/, db/schema.sql.

• Python documentation: https://docs.python.org/3/

• Flask documentation: https://flask.palletsprojects.com/

• PostgreSQL documentation: https://www.postgresql.org/docs/

• Hypothesis documentation: https://hypothesis.readthedocs.io/

• psycopg2 documentation: https://www.psycopg.org/docs/